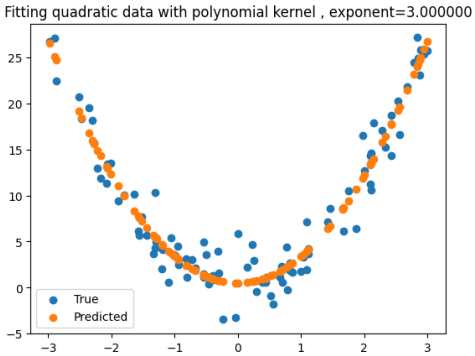
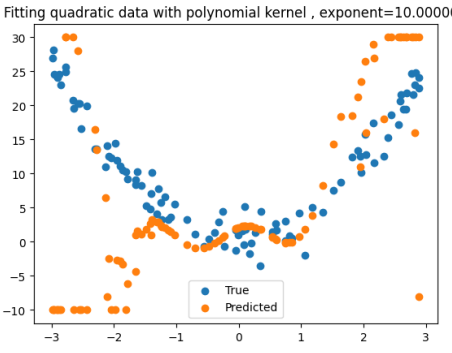
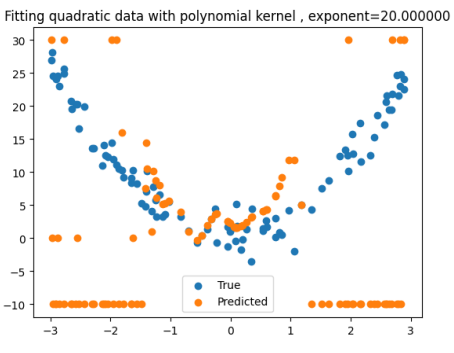
Zero Edoardo, SMID

Laboratorio 4 : Kernel Regularized Least Squares

Nel laboratorio 4 si è studiata un'estensione del problema dei minimi quadrati regolarizzati (RLS) applicato a dati di regressione in cui la relazione tra la funzione target *Y* e i dati *input X* non è lineare.
Come primo passo è stata definita la funzione *quadratic_data_gen* per generare dati quadratici della forma: *Y = X^2 w + noise* .
In seguito è stato allenato un regressore lineare per approssimare i dati quadratici di input, utilizzando le funzioni *rls_train* e *rls_predict*, ottenendo risultati insoddisfacenti perché il modello lineare non riesce a catturare la curvatura della quadratica, la predizione che si otterrà sarà una retta passante per il "vertice" della parabola dei dati e quindi non "fittando" bene i dati .
Un modo semplice per linearizzare il problema è applicare una trasformazione ai dati utilizzando una *feature map* , in particolare si è aggiunta una dimensione ai dati di input concatenando al vettore x la usa componente quadratica. La *feature* che è stata considerata è: $x \rightarrow \varphi(x)=[x,x^2]$, così facendo la predizione sui dati sarà $\boldsymbol{y= w_1x + w_2x^2= w^t \varphi(x)}$. Quindi si riesce a convertire un problema non lineare in uno lineare rispetto ai parametri *w* e quindi si è riuscito ad approssimare meglio i dati, questo approccio è efficace in casi in cui ,come questo, i dati di input sono non lineari in maniera semplice.
Successivamente è stata introdotta la *matrice Kernel* attraverso la funzione : *kernel_matrix(X1, X2, kernel_type, param)* , è un oggetto molto efficace , gioca un ruolo fondamentale nel risolvere problemi di minimi quadrati con qualsiasi forma di dati di input.In particolare sono stati studiati tre tipi di implementazioni diverse per la matrice kernel: lineare, polinomiale, gaussiana . Generalizzando un problema di *RLS* con la matrice kernel , si ottiene un problema della forma : $(K + n\lambda I)w = Y$, con *K matrice Kernel*.
Successivamente si sono implementate due funzioni per la risoluzione di un problema di *KRLS* : *krls_train* e *krls_predict* , la prima calcola il valore di *w* la seconda la predizione sulle *y* . Si sono generati dei dati quadratici e attraverso l'utilizzo della matrice kernel e delle due nuove funzioni si sono studiati i risultati.
Testando la versione lineare della matrice kernel, si ottengono gli stessi risultati del regressore lineare usato all'inizio, confermando l'inadeguatezza per i dati quadratici. Si è passato quindi al kernel polinomiale per ottenere risultati migliori , osservando anche al variare dell'esponente l'effetto sui dati in termini di *bias-varianza* e successivamente fissando l'esponente si è studiato l'effetto del parametro di regolarizzazione.
Ricordando che la funzione kernel polinomiale prende in input due matrici X1 e X2 restituisce la forma (X1 @ X2.T +1)**exponent .



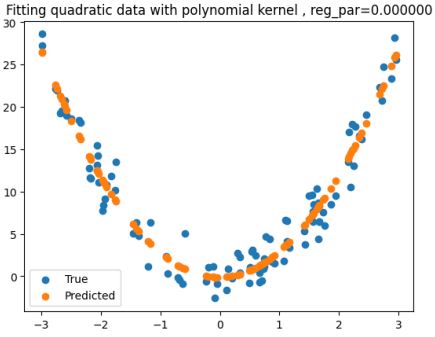


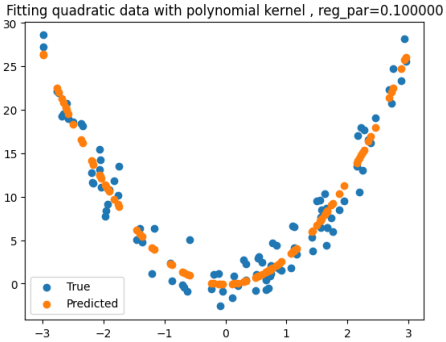


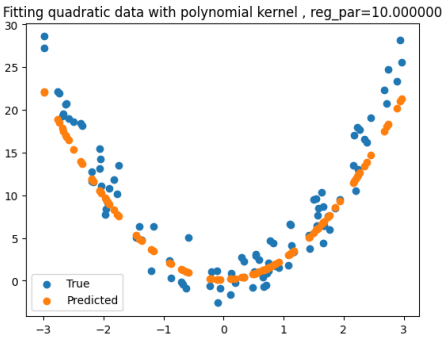
Come si può osservare dai grafici , la scelta dell'exponent è fondamentale per una predizione corretta dei dati, in generale in tutti i tre i casi i valori predetti per le x_tr tra [-1,1] sono abbastanza buoni , mentre agli estremi all'aumentare del parametro si otterranno valori sempre più elevati a causa dell'aumento del valore

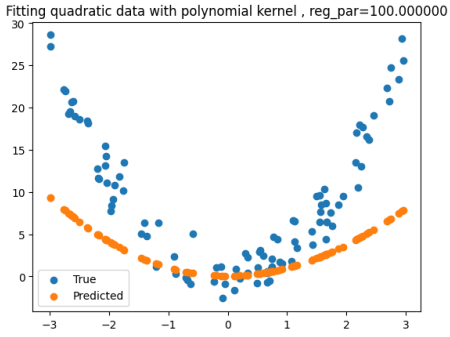
dell'esponente (i valori nei grafici sono *clippati* : i punti che assumono valore -10 o 30 in realtà hanno valori molto più alti).

Fissato il parametro di kernel si è passato a studiare l'effetto del parametro di regolarizzazione sul modello.



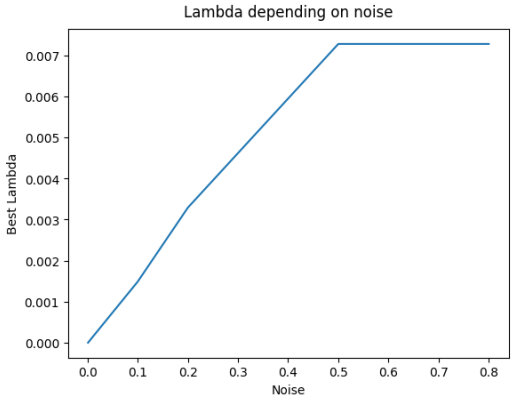






Con valori bassi del parametro di regolarizzazione , la matrice $K+reg_par \cdot n \cdot I$ si comporta similmente alla matrice kernel K, il modello cerca di adattarsi esattamente al modello di training minimizzando l'errore e ciò ha come conseguenza l' *ovrerfitting* dei dati, come si può vedere nel primo grafico. Con il crescere del parametro : reg_par=10 , 100 ecc. si osserva che la matrice $reg_par \cdot n \cdot I$ domina su quella di Kernel riducendo l'influenza dei dati, il modello diventa molto rigido e non si adatta bene ai dati di training, le predizioni inizieranno a diventare lineari o costanti (come nel caso di reg_par = 10000).

In seguito sono stati studiati dataset generati da funzioni lineari più complesse e implementando le funzioni di *cross-validation* si sono trovati il parametro di kernel per la matrice gaussiana e il parametro di regolarizzazione migliori per la *KRLS*, cioè quelli che minimizzano l'errore di validazione, quindi si sono trovati gli iperparametri migliori per predire sul *Test Set*.
Provando diverse funzioni di diversa complessità non lineare e verificando l'errore sui dati predetti rispetto a quelli di test , si può concludere che l'utilizzo della forma gaussiana e l'utilizzo dei migliori parametri trovati tramite *C-V* sono un'ottima combinazione per fare predizioni su dataset generati da funzioni complesse.
Infine si è studiato l'effetto del rumore dei dati sulla scelta del migliore parametro di regolarizzazione.



Osservando il grafico si può concludere che il valore del migliore *lambda* aumenta con la crescita del rumore sui dati, fino a stabilizzarsi quando il *noise* è maggiore di 0.5 questo accade perché il modello non è in grado di distinguere quali siano i valori reali e quelli soggetti da rumore e di conseguenza , come compromesso, fissa il valore.